

# OMI Object Model

## The Omi (o---o) Model of Bounded Agreement

```
o---o  
omi---imo
```

OMI is the **Omicron Object Model**: a bounded object doctrine for turning open-world declarations into validated, replayable, inspectable state.

It begins with the simplest possible enclosure:

```
omi---imo
```

or symbolically:

```
o---o
```

The two sides are not merely decorative.

They are the bookends of a relation.

The hyphen field between them is not empty space.

It is the join field.

It is the bus.

It is the gravity bridge.

It is the meta-compiler lane.

It is where a declaration becomes an inspectable object.

The zero-point of the model is:

```
 $\Omega_0 = 0! = \text{omi---imo}$ 
```

From this zero-point, OMI defines a full pipeline:

```
validate -> resolve -> record -> materialize -> project -> inspect
```

The forbidden shortcut is:

```
render -> believe
```

OMI exists to prevent that shortcut.

A thing may render in a browser, appear in a DOM node, show up as a card, exist as a QR code, or travel through gossip. None of those make it accepted.

The reader may recognize.

The resolver may promote.

Only validation and receipt may accept.

---

## 1. The Core Promise

OMI lets people save, share, verify, and move objects without losing history or context.

It is not only a protocol.

It is not only a notation.

It is not only a document model.

It is a way to say:

```
This object has a boundary.  
This object has a transition.  
This object has a receipt.  
This object can be replayed.  
This object can be projected.  
This object can be inspected.
```

OMI is built for open-world systems where no single platform owns truth.

A normal platform says:

```
The app owns the object.  
The database owns the state.
```

```
The server decides the current version.  
The UI decides what the user sees.
```

OMI says:

```
The frame must validate.  
The transition must resolve.  
The receipt must record.  
The projection must remain downstream.  
The user must be able to inspect.
```

The difference is fundamental.

In OMI, projection is not authority.

```
carrier != authority  
projection != validation  
prefix != identity  
gossip != authority  
render != believe
```

---

## 2. The Object Boundary

The readable mnemonic boundary is:

```
omi---imo
```

The symbolic lower boundary uses Omicron anchors:

```
o = 0x03BF = Greek small omicron  
O = 0x039F = Greek capital Omicron
```

These are the structural anchors of the frame.

They are not typography.

They are not styling.

They are not reader decoration.

They define the lower object boundary.

The mnemonic reading is:

```
omi  = entry / low gate / declaration side  
imo  = exit / high gate / closure side  
---  = join field / bridge / compiler lane
```

The deeper reading is:

```
omi enters.  
--- joins.  
imo closes.
```

The hyphen is the operation.

In ordinary language, a hyphen joins terms:

```
open-world  
peer-to-peer  
meta-compiler
```

In OMI, the hyphen joins address terms into a bounded instruction word:

```
omi-ip4-gravity-ip6-imo
```

This does not mean every string containing `omi` is an OMI object.

A string containing `omi` may be plain text.

An `omi-*` token may be a readable alias.

An `id="omi-"` attribute may be a DOM projection carrier.

A `data-omi` attribute may be a CSSOM/JSDOM bridge.

None of these are accepted identity by themselves.

The production rule is:

```
omi-* / imo-* = alias or template until resolved.  
Only accepted frames become OMI objects.
```

## 3. The Authority Pipeline

The OMI authority pipeline is:

```
reader -> resolver -> validation -> receipt -> materializer -> projection ->  
inspection
```

Each layer has a different job.

### Reader

The reader recognizes notation.

It may see:

```
omi---imo  
omi-<tail>  
imo-<tail>  
8-segment frame candidate  
DOM carrier  
CSSOM carrier  
plain text
```

But recognition is not acceptance.

```
reader may recognize
```

### Resolver

The resolver promotes a recognized form into a candidate.

It may say:

```
This looks like an OMI frame candidate.  
This looks like a projection alias.
```

This looks like a received edit.  
This looks like a recoverable gossip candidate.

But promotion is not acceptance.

resolver may promote

## Validation

Validation checks structure.

The canonical frame must satisfy:

$Q\_frame(S) = 0$

If the frame fails, it rejects.

## Resolution

A valid frame yields a truth row:

(LL, NN, MM)

The resolver must prove that **MM** is reachable from **NN** under the accepted Delta/Fano path.

## Receipt

A successful transition writes a receipt.

The receipt is the acceptance boundary.

Only validation and receipt may accept.

## Materializer

The materializer reads accepted envelopes and applies registered rule blocks.

The materializer may know about DOM, CSSOM, JSDOM, OMI object patches, snub geometry, projection control, and portal surfaces.

The gate does not.

## Projection

Projection displays accepted state.

Projection may use:

```
DOM
CSSOM
JSDOM
SVG
Canvas
JSON Canvas
QR
A-Frame
portal cards
document views
```

Projection must never become authority.

## Inspection

Inspection explains what happened.

It shows:

```
receipts
replay
history
source
context
conflicts
gossip candidates
raw frame
```

---

## 4. The 128-Bit Frame

An OMI frame is an 8-segment structure:

S0-S1-S2-S3-S4-S5-S6-S7

Each segment is 16 bits.

Together:

$8 \times 16 = 128 \text{ bits}$

A canonical frame has this shape:

S0 = 0xLL00  
S1 = 0x03BF  
S2 = 0xNNNN  
S3 = 0x2BLL  
S4 = 0x2FLL  
S5 = 0xMMMM  
S6 = 0x039F  
S7 = 0xLLFF

The frame contains:

LL = lens / scope / ket selector  
NN = antecedent  
MM = consequent

The Omicron anchors are:

S1 = 0x03BF = o  
S6 = 0x039F = 0

The repeated LL positions create a variance lock:

S0, S3, S4, S7

The transition pair is:

S2 = NN  
S5 = MM



So the frame anatomy is:

```
boundaries -> S1, S6  
modulator  -> S0, S3, S4, S7  
transition -> S2, S5
```

A frame is valid only if:

```
Q_frame(S) = E_var + E_const = 0
```

The frame does not pass because it looks right.

It passes because the structural equation closes.

---

## 5. The Truth Row

A valid frame yields exactly one truth row:

```
(LL, NN, MM)
```

The truth row is the minimal agreement face.

It can be read as:

```
lens / antecedent / consequent
```

or:

```
scope / beginning / resolution
```

or:

```
declaration / definition / receipt
```

The important relation is:

NN -> MM

NN is the antecedent state.

MM is the consequent state.

LL does not prove the transition.

LL scopes where the accepted transition lands.

The transition is accepted only if the Delta/Fano resolver can reach MM from NN within the bounded window.

Short form:

NN begins.  
MM resolves.  
LL scopes.

---

## 6. The Delta Law

The Delta Law is the atomic transition engine.

A simple form is:

$\Delta\_C(x) = \text{rotr}(x,1) \text{ xor } \text{rotr}(x,3) \text{ xor } \text{rotr}(x,2) \text{ xor } C$

Bounded to 16 bits:

$\Delta_{16\_C}(x) = \Delta\_C(x) \& 0xFFFF$

The Delta Law uses bitwise operations:

rotation  
xor  
masking  
constant modulation

This keeps the lower substrate integer-bound, deterministic, and replayable.

The resolver attempts:

```
NN -> Δ(NN) -> Δ2(NN) -> ... -> MM
```

within a bounded Fano window.

If **MM** is reached, the transition is accepted.

If not, the frame rejects.

The short doctrine:

```
Delta proves.  
Fano bounds.  
Receipt records.
```

---

## 7. Fano Resolution

The Fano layer gives OMI a bounded finite incidence selector.

A Fano plane has:

```
7 points  
7 lines  
3 points per line
```

OMI reads this as a finite triangular incidence field.

The truth row:

```
(LL, NN, MM)
```

is triangular.

The Fano layer asks:

Which relation closes?  
Which triangle resolves?  
Which line carries the accepted path?

This is sometimes called the “Fano lottery,” but it is not random.

It is finite incidence selection.

The Fano layer prevents the system from searching an unbounded space.

It gives a bounded window for resolution.

Short form:

Fano selects the triangle.  
Delta proves the path.  
Receipt remembers the result.

---

## 8. LL as Ket

The most precise role of **LL** is:

LL is the ket-like scope selector of the frame.

In Dirac notation, a ket such as:

$|\psi\rangle$

represents a state vector.

OMI does not use quantum mechanics literally, but the analogy is useful.

The truth row may be read as:

$(|LL\rangle, NN, MM)$

or:

scope ket + antecedent + consequent

LL is not the object body.

LL is not the full meaning.

LL is not geometry.

LL is the scoped selector applied to the accepted transition.

It tells the materializer where the proven transition lands.

Short form:

```
LL scopes.  
NN begins.  
MM resolves.
```

---

## 9. The LL Nibble Split

LL is a byte.

It can be split into two nibbles:

LL = 0xAB

Where:

```
A = high nibble  
B = low nibble
```

OMI reads this as:

```
high nibble = context / lens family / carry-forward scope  
low nibble  = execution slot selector
```

The POSIX gate computes:

```
slot_index=$((ll & 0x0f))  
rule_slot=$((0xe0 + slot_index))  
context=$((ll >> 4) & 0x0f))
```

This is pure bitwise routing.

No semantic branch is required.

The gate does not ask:

```
Is this a snub operation?  
Is this a DOM patch?  
Is this a viewport translation?  
Is this a rotation?
```

The gate only extracts:

```
context  
slot_index  
rule_slot
```

The materializer registry owns the meaning.

---

## 10. The Folded 0xE\_ Execution Manifold

The completed materializer band is:

```
0xE0-0xEF
```

It has 16 slots.

The low nibble of `LL` selects one of them:

```
rule_slot = 0xE0 + (LL & 0x0F)
```

This replaces the earlier `LL mod 12` model.

`LL mod 12` described the twelve inner packed materializer spheres.

The final substrate uses the complete low nibble to saturate the whole hexadecimal band.

The mapping is:

```
0x0 -> 0xE0
0x1 -> 0xE1
0x2 -> 0xE2
0x3 -> 0xE3
0x4 -> 0xE4
0x5 -> 0xE5
0x6 -> 0xE6
0x7 -> 0xE7
0x8 -> 0xE8
0x9 -> 0xE9
0xA -> 0xEA
0xB -> 0xEB
0xC -> 0xEC
0xD -> 0xED
0xE -> 0xEE
0xF -> 0xEF
```

The first twelve slots are:

```
0xE0-0xEB = inner materializers
```

The last four slots are:

```
0xEC-0xEF = outer controller vertices
```

Together:

```
0xE0-0xEF = one accepted execution manifold
```

---

## 11. The First Twelve: Inner Materializers

The first twelve slots pack the object.

```
0xE0-0xEB pack the object.
```

They correspond to the inner materializer field.

A registry may define them as:

```
0xE0 = page-frame-root  
0xE1 = DOM import/export  
0xE2 = CSSOM import/export  
0xE3 = JSDOM normalization  
0xE4 = OMI object patch  
0xE5 = edge traversal  
0xE6 = vertex traversal  
0xE7 = snub bridge  
0xE8 = stellated octahedron mirror  
0xE9 = Fano triangle selection  
0xEA = 59/60/61 face band  
0xEB = receipt projection/export
```

These are not shell branches.

They are accepted rule addresses.

The shell does not know what any of these mean.

The registry knows.

---

## 12. The Last Four: Platonic Controller Vertices

The final four slots hold the object.

```
0xEC-0xEF hold the object.
```

They are not a separate UI command channel.

They are the final four addresses of the same accepted execution manifold.

They form the outer controller surface:

```
0xEC = translation / viewport / focus  
0xED = scale / recursion depth
```



```
0xEE = rotation / spin / orientation
0xEF = boundary / opacity / clipping
```

Geometrically, they can be read as Platonic controller vertices folded into the execution band.

The first twelve are the Archimedean/Catalan materializer field.

The final four are the Platonic controller vertices of the simplex closure.

Short form:

```
The first twelve pack.
The last four hold.
```

The controller slots must still pass through:

```
Q_frame validation
Delta/Fano resolution
receipt writing
```

No controller exists outside receipt.

---

## 13. No Separate Controller Channel

OMI rejects separate command channels for lens control.

A separate channel would create double authority.

It would allow a viewport, rotation, scale, or clipping rule to move outside the accepted receipt path.

That would reintroduce the forbidden shortcut:

```
render -> believe
```

The rule is:

```
No controller outside receipt.
```

A local browser may always pan, zoom, scroll, or rotate something as local UI behavior.

But a conforming OMI projection must not treat that local behavior as accepted state.

The stricter rule is:

The viewport is receipt-bound when controlled through 0xEC-0xEF.  
A conforming projection must not treat out-of-band camera or lens changes as accepted state.

This preserves the difference between:

local display behavior

and:

accepted projection state

---

## 14. The Gate and the Materializer

The lower gate is policy-blind.

The materializer owns semantics.

### Gate

The gate does:

```
parse frame
validate Q_frame(S) = 0
extract (LL, NN, MM)
resolve NN -> MM
derive context
derive slot_index
derive rule_slot
write receipt
emit envelope
```

The gate does not:

```
know snub geometry
know Platonic solids
know Archimedean/Catalan duals
know DOM policy
know CSSOM policy
know JSDOM policy
know portal UX
know A-Frame policy
know application meaning
```

## Materializer

The materializer does:

```
read accepted envelope
lookup rule_slot
apply registered implementation
respect context
respect steps
respect chirality
project result
explain result
```

The split is:

```
start.sh is the gate.
The registry is the materializer.
```

The final substrate doctrine:

```
LL selects.
Delta proves.
Fano bounds.
Receipt authorizes.
Rule block materializes.
Projection displays.
```

---

## 15. Execution Envelope

An accepted frame emits an execution envelope.

Example:

```
{  
  "accepted": true,  
  "LL": "0x3E",  
  "context": "0x3",  
  "slot_index": "0xE",  
  "rule_slot": "0xEE",  
  "rule_band": "0xE0-0xEF",  
  "NN": "0x7c00",  
  "MM": "0x1434",  
  "steps": 3,  
  "chirality": "right",  
  "receipt": "..."  
}
```

The materializer reads:

```
context = 0x3  
rule_slot = 0xEE  
steps = 3  
chirality = right  
receipt = accepted
```

The registry may say:

```
0xEE = rotation-spin
```

Then the materializer applies the registered operation.

The shell never knows that `0xEE` means rotation.

It only knows:

```
the frame was valid  
the transition resolved  
the receipt was written  
the rule address was emitted
```

## 16. Steps and Chirality

The execution envelope gives the materializer four orthogonal parameters:

```
context
rule_slot
steps
chirality
```

Their roles are:

```
context    = lens family / carry-forward scope
rule_slot  = operation address
steps      = magnitude / phase / recursion depth
chirality  = handed direction
```

Example:

```
rule_slot = 0xEE
context   = 0x3
steps     = 3
chirality = right
```

The materializer may interpret this as:

```
rotate the projection under context 0x3 by three accepted steps in the right-
handed direction
```

But this is materializer policy.

It is not gate policy.

---

## 17. Carry-Forward Scope

The high nibble of `LL` carries forward context.

```
context = (LL >> 4) & 0x0F
```

If the active scope is represented as a 4-bit value, then the next scope may be derived by XOR:

```
next_scope = current_scope xor context
```

or:

```
scope' = scope ⊕ high_nibble(LL)
```

This gives OMI a bounded carry-forward model.

The carry-forward scope is:

```
bitwise  
bounded  
replayable  
low-level  
compatible with the frame
```

The shell may emit context.

The materializer may apply scope.

The receipt records what happened.

---

## 18. The Quadratic Object Model

The geometric object model is governed by:

$$Q_{xy}(x,y) = 60x^2 + 16xy + 4y^2$$

This is not only a formula.

It is a structural lifecycle.

### **4y<sup>2</sup>: Platonic Seed**

The low term is the atomic seed.

$4y^2$  = Platonic seed / local declaration / vertex body

It represents the first stable body of declaration.

## 16xy: Archimedean/Catalan Bridge

The middle term is the hyphen field.

$16xy$  = snub bridge / chiral transformation / relation bus

This is where the object is joined, twisted, dualized, and re-faced.

This is where snub truncation lives.

## 60x<sup>2</sup>: 4D Regular Orbit

The high term is the cosmic orientation field.

$60x^2$  = face traversal / sexagesimal sweep / 4D regular closure

This is where the accepted object gains broader orientation and replayable consistency.

Short form:

$4y^2$  seeds.  
 $16xy$  snubs.  
 $60x^2$  sweeps.

---

## 19. Snub Truncation as the OMI Operation

OMI does not overwrite objects.

OMI snubs them.

Snub truncation does three things:

```
cut
twist
re-face
```

In OMI:

```
cut      = isolate unresolved boundary
twist    = introduce chirality through the bridge
re-face  = create triangular agreement surfaces
record   = preserve the transformation path
project  = expose the accepted state
```

The snub operation is better than permutation or circular inversion because it transforms the body while preserving its relation to the parent.

Permutation only reorders.

Circular inversion only reverses inside and outside.

Snubbing:

```
cuts excess
twists relation
creates handedness
adds triangular incidence
preserves parent recoverability
```

This is the geometry of the hyphen.

```
omi enters.
--- snubs.
imo closes.
```

---

## 20. Snub Cube and Snub Dodecahedron

The snub cube and snub dodecahedron are the two clearest witnesses of OMI chirality.



## Snub Cube

The snub cube begins from the cube.

It preserves:

6 square faces

It adds:

32 triangular faces

It has:

38 total faces  
24 vertices  
60 edges

OMI reads it as:

snub cube = edge traversal witness

## Snub Dodecahedron

The snub dodecahedron begins from the dodecahedron.

It preserves:

12 pentagonal faces

It adds:

80 triangular faces

It has:

92 total faces  
60 vertices  
150 edges

OMI reads it as:

snub dodecahedron = vertex traversal witness

Together:

snub cube walks edges  
snub dodecahedron walks vertices  
 $60x^2$  walks faces

This gives OMI a traversal triad:

edge traversal  
vertex traversal  
face traversal

---

## 21. Stellated Octahedron and Mirror Space

The inverse of the local snub space can be modeled by the stellated octahedron.

A stellated octahedron can be read as two interpenetrating tetrahedra.

This gives:

tetrahedron A  
tetrahedron B  
shared centroid  
mirror orientation

In OMI:

stellated octahedron = inverse mirror around the tetrahedral centroid

The snub pair gives the P2P traversal.

The stellated octahedron gives the mirror-local inverse.

At the face diagonal:

$\sqrt{2}$

local traversal crosses into canonical mirror space.

The 59/60/61 band stabilizes the sexagesimal center:

59 = approach  
60 = center  
61 = exit

Short doctrine:

Snub cube walks edges.  
Snub dodecahedron walks vertices.  
60x<sup>2</sup> walks faces.  
The stellated octahedron mirrors local space.  
 $\sqrt{2}$  opens the face crossing.  
59/60/61 stabilizes the center.

---

## 22. MCRSGSP: Gossip Storage Substrate

OMI can use a gossip storage substrate for distributed recoverability.

That substrate is MCRSGSP:

Monotone Causal Reed-Solomon Gossip Storage Protocol

Its role is:

Gossip moves.  
Reed-Solomon recovers.  
Causality bounds.  
OMI accepts.  
Projection displays.

MCRSGSP does not decide accepted objects.

It provides recoverable candidates.

OMI decides accepted objects.

The boundary is:

```
MCRSGSP provides recoverable candidates.  
OMI decides accepted objects.
```

This prevents the shortcut:

```
gossip -> authority
```

Fragments may propagate.

Reed-Solomon may reconstruct.

Causality may bound a candidate.

But acceptance still requires:

```
Q_frame validation  
Delta/Fano resolution  
receipt
```

---

## 23. DOM, CSSOM, JSDOM, and the OMI Object Model

OMI treats browser surfaces as projections and carriers.

The document workbench should support four synchronized surfaces:

```
DOM  
CSSOM  
JSDOM  
OMI Object Model
```

Their roles are:

```
DOM    = live browser projection
CSSOM  = style and reader lens projection
JSDOM  = portable/headless normalized document
OOM    = accepted object state, receipts, replay, trust
```

The doctrine:

```
DOM renders.
CSSOM styles.
JSDOM normalizes.
OMI accepts.
Receipts remember.
Exports move.
Imports return.
```

A DOM edit is a candidate.

A CSSOM change is a projection change.

A JSDOM snapshot is a portable reconstruction.

An OMI object is accepted only after validation and receipt.

Import flow:

```
import -> recognize -> candidate -> review -> validate -> receipt
```

Receive flow:

```
receive -> candidate -> compare -> resolve -> validate -> receipt
```

Export flow:

```
projection export = what it looks like
object export     = what was accepted
receipt export    = why it is trusted
```

## 24. User Interface Model

The user interface should not start as a protocol dashboard.

It should start as an object workspace.

The four user-facing modes are:

```
Focus
Attention
Observation
Experience
```

### Focus

What object am I working with?

```
current object
summary
status
primary action
```

### Attention

What needs review?

```
unverified imports
received edits
conflicts
pending mirrors
local-only changes
```

### Observation

Why should I trust this?

```
receipts
replay
source
context
```

validation status  
gossip candidates

## Experience

What happened over time?

history  
shared objects  
joined events  
exports  
mirrors  
accepted edits

Short doctrine:

Focus selects.  
Attention alerts.  
Observation verifies.  
Experience remembers.

The main interface should be an Object Inbox.

Normal users should see:

objects  
trust labels  
history  
receipts  
actions

Power users may inspect:

frames  
truth rows  
rule slots  
gossip candidates  
raw OMI

## 25. Trust Labels

OMI objects should expose plain trust states:

```
Accepted
Unverified
Conflict
Rejected
Local-only
Mirrored
Pending
Replayed
```

A card might say:

```
Sunday Night Lights Ride
Event
Status: Accepted
Receipts: 3
Mirrors: 2
Conflicts: none
```

An unverified object might say:

```
Status: Unverified
Receipts: none
Source: unknown
Open read-only?
```

A conflict might say:

```
Status: Conflict
Accepted versions: 2
Compare / Keep both / Fork / Resolve
```

OMI should not hide disagreement.

It should make disagreement inspectable.

```
disagreement -> missing structure -> new object/path/test/receipt
```



---

## 26. A-Frame and Visualization Surfaces

A-Frame, Canvas, SVG, DOM, and other visual systems are projection surfaces.

They may render accepted state.

They must not participate in accepting state.

The doctrine:

```
A visualization dependency may render accepted state,  
but it must never participate in accepting state.
```

This means:

```
A-Frame = projection only  
DOM = projection carrier  
CSSOM = reader lens  
SVG = projection surface  
Canvas = projection surface
```

None of them replace:

```
Q_frame  
Delta/Fano  
receipt
```

This preserves the boundary:

```
render -> believe
```

is forbidden.

---

## 27. Receipts and Replay

A receipt records an accepted transition.

The replay ring gives OMI memory.

A canonical ring size is:

7! = 5040

The receipt may contain:

LL  
NN  
MM  
steps  
chirality  
rule\_slot  
context  
slot\_index  
source  
checksum  
signature

The receipt lets the system answer:

What happened?  
Which transition was accepted?  
Where did it materialize?  
Which rule slot was selected?  
Which context was active?  
Which direction was used?  
Can this be replayed?

A receipt is not just metadata.

It is the accepted memory of the object.

Short doctrine:

Receipt closes.  
Replay remembers.  
Inspection explains.

## 28. Full Stack Summary

The full OMI stack is:

```
Reader
  recognizes notation

Resolver
  promotes candidates

MCRSGSP
  recovers fragments and candidates

Q_frame
  validates structure

Delta/Fano
  proves transition and bounds path

Receipt
  accepts and records

OxE_Materializer
  applies accepted rule address

Projection
  displays accepted state

Inspection
  explains evidence and history
```

Short stack:

```
Reader recognizes.
Resolver promotes.
MCRSGSP recovers.
OMI validates, resolves, and receipts.
Rule block materializes.
Projection displays.
Inspection explains.
```

Forbidden shortcuts:

```
prefix -> identity  
gossip -> authority  
render -> believe  
controller -> outside receipt
```

## 29. The Final Substrate Canon

The OMI substrate is sealed by the `0xE_` execution band.

```
0xE0-0xEB pack the object.  
0xEC-0xEF hold the object.
```

The low nibble of `LL` selects the slot:

```
rule_slot = 0xE0 + (LL & 0x0F)
```

The high nibble carries context:

```
context = (LL >> 4) & 0x0F
```

The transition pair remains:

```
NN -> MM
```

Delta/Fano proves it.

The receipt authorizes it.

The registry materializes it.

The projection displays it.

The inspector explains it.

Final doctrine:

```
LL scopes.  
NN begins.
```

```
MM resolves.  
Delta proves.  
Fano bounds.  
The high nibble carries.  
The low nibble selects.  
The first twelve pack.  
The last four hold.  
Receipt closes.
```

Final one-line canon:

```
LL is the ket of the frame: a 4-bit carry-forward scope selector whose low  
nibble lands the accepted NN->MM transition inside the folded 0xE_ materializer  
manifold.
```

---

## 30. Final Statement

OMI is a bounded object model for declared activity.

It begins with:

```
omi---imo
```

It validates structure through:

```
Q_frame(S) = 0
```

It resolves transition through:

```
Delta/Fano
```

It records acceptance through:

```
receipt
```

It materializes execution through:

0xE0-0xEF

It projects through:

DOM / CSSOM / JSDOM / SVG / Canvas / Portal

It explains itself through:

inspection / replay / receipts / context

The object is not the rendering.

The object is not the prefix.

The object is not the gossip candidate.

The object is the accepted, replayable relation.

Short canon:

o---o encloses.  
LL scopes.  
NN begins.  
MM resolves.  
Delta proves.  
Fano bounds.  
Receipt closes.  
0xE\_ materializes.  
Projection displays.  
Experience remembers.

One-line product promise:

OMI lets people save, share, verify, and move objects without losing history or context.

One-line substrate promise:

The OMI object model turns open-world declarations into bounded, validated, replayable, and inspectable agreement.